

Chapter 5

Static Logic Gates

One extremely powerful aspect of CMOS is the ability to create single gate circuits that can implement functions consisting of several basic logic operations. This makes digital CMOS design quite different from classical logic design techniques, since now the logic expressions and the corresponding circuits become very closely related. A **static logic gate** is one that has a well defined output once the inputs are stabilized and the switching transients have decayed away. Static CMOS logic gates are relatively easy to design and use. This chapter deals with the static logic gates, from simple NAND and NOR operations to complex functions that are quite large and powerful.

5.1 Complex Logic Functions

Complex logic gates provide functions that consist of several primitive NOT, AND, or OR operations. Consider a three-variable function with input variables a , b , and c . Suppose that we construct the function

$$f(a, b, c) = \overline{a \cdot \bar{b} \cdot c + \bar{a} \cdot b \cdot c + a \cdot b \cdot c} \quad (5.1)$$

this is an example of a canonical AOI (and-or-invert) equation¹ which falls into the category of a complex logic operation. If we reverse the ordering of the logic operations, we obtain an OAI (or-and-invert) expression. For example,

$$g(a, b, c) = \overline{(a + \bar{b} + c) \cdot (\bar{a} + b + c) \cdot (a + b + c)} \quad (5.2)$$

exhibits this characteristic, and is a complex logic form. It is worthwhile to note that f and g are

¹ Recall that the “AND” operation has precedence over the “OR” operation.

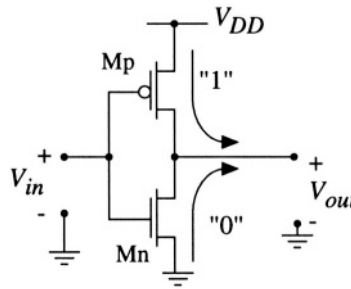


Figure 5.1 Operation of the CMOS inverter circuit

duals of each other, i.e., interchanging the AND and OR operations in one gives the other.

Complex logic gates are constructed using the CMOS inverter as a basis. As discussed in the previous chapter, the nFET and pFET are placed to act as pass transistors. With regards to the circuit shown in Figure 5.1, the input voltage V_{in} controls the conduction modes of both transistors. When $V_{in} = 0\text{V}$, the pFET conducts the power supply voltage V_{DD} to the output. When $V_{in} = V_{DD}$, the nFET is ON, and transmits the ground (0V) to the output. This provides the foundation for creating logic gates using arrays of FETs. Inputs are used to control whether V_{DD} or ground is connected to the output. As with the inverter, only one conduction path can exist at a time. This specifically eliminates the possibility of both the power supply and ground being simultaneously connected to output. The opposite case where neither is connected to the output is not desirable in a logic gate, but is useful for isolating the circuit; this will lead to the concept of a “tri-state” logic gate later in the chapter.

In order to construct a complex logic gate, let us replace the single inverter nFET by an array of nFETs that are connected to operate as a large switch. Similarly, we will substitute an array of pFETs for the single pFET used in the inverter, and view the pFET array as a “giant” switch. In order to insure proper electrical operation, however, we must exercise care so that operation of the nFET array complements the operation of the pFET array. This means that if one array is a closed switch, the other must be open.

The general structure of a complex logic gate can be created by the following steps.

- Provide a complementary pair (an nFET and a pFET with a common gate) for each input;
- Replace the single nFET with an **array** of nFETs that connects the output to ground;
- Replace the single pFET with an **array** of pFETs that connects the output to V_{DD} ;
- Design the nFET and pFET switching network so that only one network acts as a closed switch for any given input combination.

This results in the general network shown in Figure 5.2 for the case of three inputs A , B , and C .

With $m > 1$ inputs, the nFET and pFET arrays are viewed as large “composite” switches, with each array containing m MOSFETs. For a given input combination, only one composite switch can be closed. If the pFET switching array is closed, then the output voltage is $V_{out} = V_{DD}$, giving a logic 1 results. Conversely, if the nFET array is closed, then the output is a logic 0 with $V_{out} = 0\text{V}$. Both cases are shown in Figure 5.3. Note that, for proper operation, the arrays must be designed so that the two cases where either (i) both switching arrays are closed, or, (ii) both switching arrays are open, cannot occur, since both situations give an undefined output.

In this chapter, we will first examine the NAND and NOR gates to see how the complementary structures work. Then, we will progress to more complex logic gates, where the integration capabilities of CMOS will become clear. As will be stressed in the treatment, CMOS provides an extremely powerful approach to building complicated digital logic networks in a very efficient

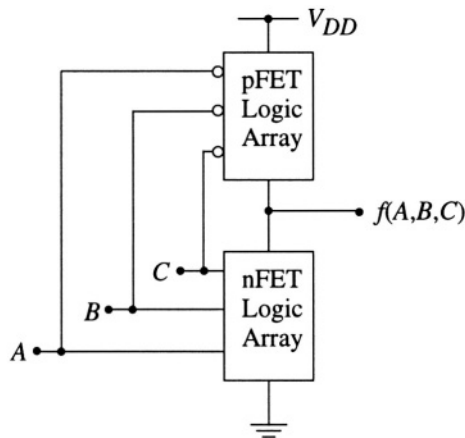


Figure 5.2 General structure of a static CMOS logic gate

manner. It allows for an amazingly large collection of circuit design styles to be developed, which provides the circuit designer with many options to choose from.

5.2 CMOS NAND Gate

The first CMOS logic gate that we will study is the 2-input NAND (**NAND2**). Figure 5.4 shows the logic symbol and the truth table where the inputs are denoted by Boolean variables A and B . The NAND2 operation is described by

$$\begin{aligned}
 f &= \overline{A \cdot B} \\
 &= \text{NOT}(A \cdot B) \\
 &= \overline{A} + \overline{B}
 \end{aligned}
 \tag{5.3}$$

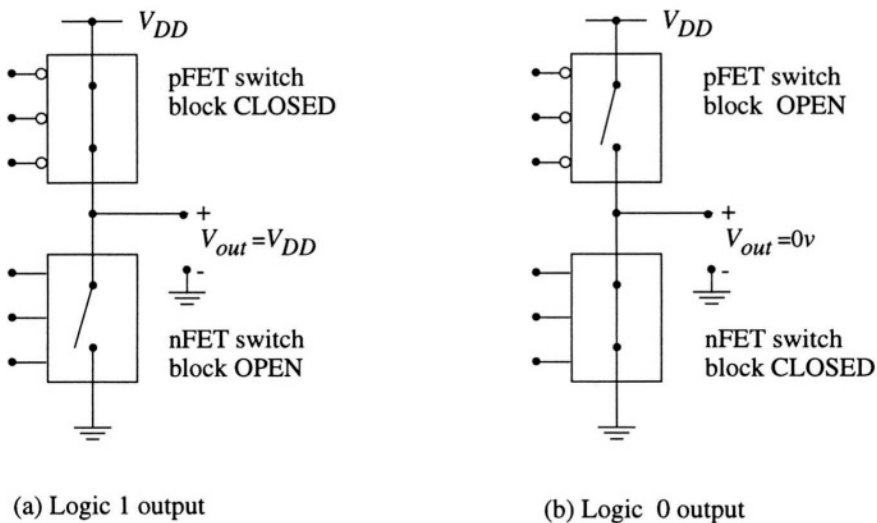


Figure 5.3 General operation using switch logic

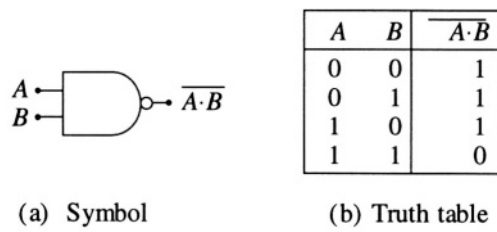


Figure 5.4 NAND2 symbol and truth table

where the last step follows from the DeMorgan rules. This provides us with the information needed to create a CMOS NAND2 gate.

To construct a CMOS circuit that provides this function we will use two complementary pairs, one for each of the inputs A and B , and create the nFET and pFET arrays according to the needed outputs. First, note that there is only a single case where the output is a 0. This occurs when both inputs are at logic 1 values. Translating this observation into voltages then says that the output voltage $V_{out} = 0v$ if and only if both of the two input voltages high, i.e., $V_{in,A} = V_{DD} = V_{in,B}$. Since the nFETs connect the output node to ground, this requires that the two nFETs be connected in series. If either input voltage is low, then $V_{out} = V_{DD}$, indicating that the output node must be connected to the power supply. To accommodate these cases, we will wire the two pFETs in parallel. Combining the requirements for the FETs results in the circuit shown in Figure 5.5.

The logical operation of the circuit can be verified by working in reverse. Consider the series-connected nFETs MnA and MnB . If both $V_{in,A}$ and $V_{in,B}$ are high, then these transistors are active and conduct current while both pFETs are in cutoff. This provides a strong conduction path to ground and gives an output voltage of $V_{OL} = 0v$. However, if either A or B is low (either individually or at the same time) then there is no path to ground; in this case, at least one p-channel device is conducting to the power supply, giving a value of $V_{OH} = V_{DD}$.

Another approach to deriving the logic function is to use the FET switching formalism introduced at the end of Chapter 4. To apply this technique, we associate the power supply voltage V_{DD}

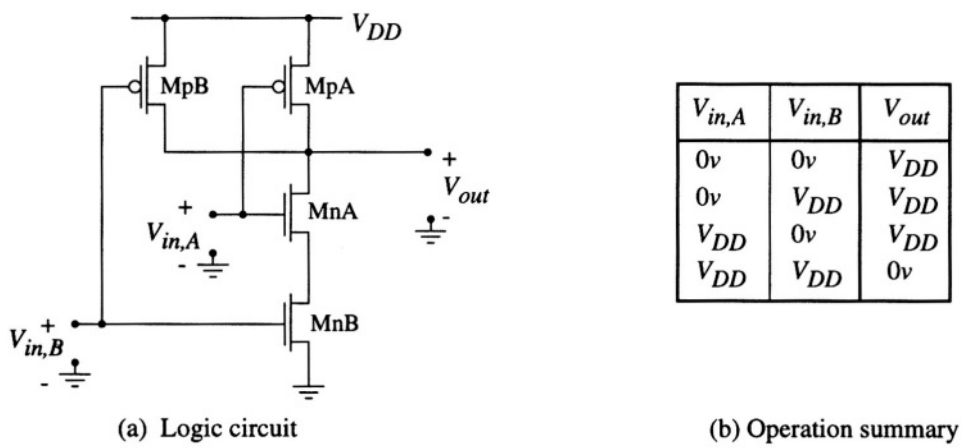


Figure 5.5 CMOS NAND2 logic gate

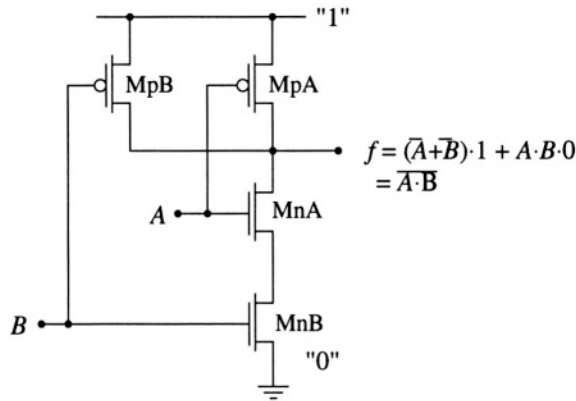


Figure 5.6 Logical operation of the NAND2 gate

with a logic 1, and the ground connection with a logic 0. Denoting the inputs by simply A and B results in the circuit shown in Figure 5.6. The output is viewed as the OR operation between the pFET switches and the nFET switches such that

$$f = \left(\overline{A} + \overline{B} \right) \cdot 1 + A \cdot B \cdot 0 \quad (5.4)$$

The first terms are obtained by ANDing the complement of the pFET gate variable with the input ("1"), while the second term represents the series nFETs that transmit the "0" to the output. Since the nFET terms are logically 0, we may apply the DeMorgan theorem to the pFET terms to arrive at

$$f = \overline{A \cdot B} \quad (5.5)$$

which is the NAND operation. The important point to be made here is that the **topology** of the circuit, i.e., the placing and wiring of the transistors, determines the logic function in its entirety.

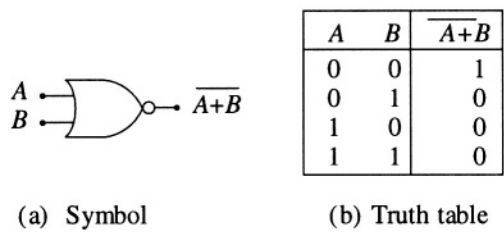


Figure 5.15 Symbol and truth table for the NOR2 gate

5.3 CMOS NOR Gate

Now let us examine a 2-input CMOS NOR gate with inputs *A* and *B*. Figure 5.15 shows the logic symbol and truth table; we will refer to this as a NOR2 gate. The operation is characterized by the by the fact that a logic 1 at either input (or both inputs) causes a logic 0 output. Logically, the NOR2 operation is denoted by

$$\begin{aligned} f &= \overline{A+B} \\ &= \overline{A} \cdot \overline{B} \end{aligned}$$

(5. 38)

where (*A+B*) denotes the OR operation between *A* and *B*; the second step follows from applying the DeMorgan rule.

A CMOS NOR2 gate can be built by using two complementary pairs as shown in Figure 5.16(a). Input *A* is connected to MnA and MpA, while *B* controls MnB and MpB. Note that the nFETs are connected in parallel, while the pFETs form a series chain. To understand the operation of the gate, we examine the conduction states of the transistors for different input voltages *V_{in,A}* and *V_{in,B}*. If *V_{in,A}*=*V_{DD}*, then MnA is ON and MpA is OFF; since MnA provides a conducting path from the ground to the output, *V_{out}* = 0v. Setting *V_{in,B}*=*V_{DD}* turns MnB ON and MpB OFF and also

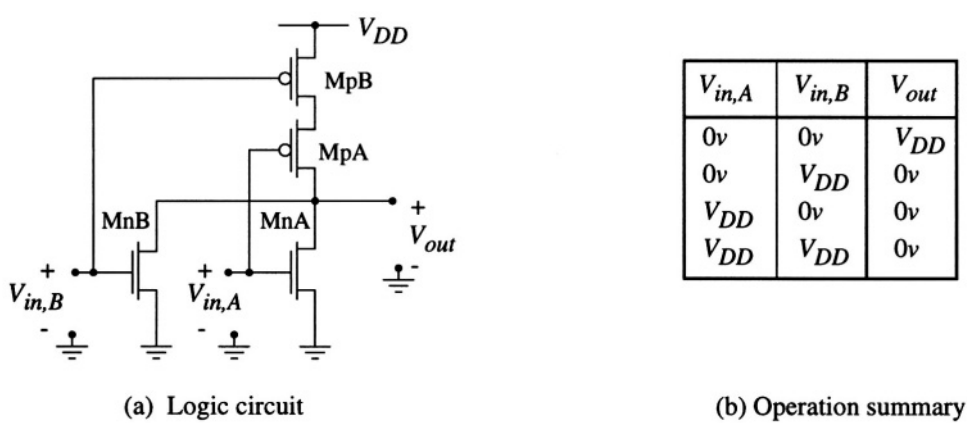


Figure 5.16 CMOS NOR2 circuit

results in $V_{out} = 0v$. And, if both $V_{in,A}$ and $V_{in,B}$ are high, then both nFETs are ON and the output voltage is $V_{out} = 0v$. The only input combination that results in $V_{out} = V_{DD}$ is when $V_{in,A} = 0v = V_{in,B}$, since both pFETs are ON while both nFETs are OFF. As verified by the truth table in Figure 5.16(b), this gives exactly the NOR2 operation.

We may also verify the logic function by viewing the circuit as a simple multiplexor between the power supply V_{DD} ("1") and ground ("0") as shown in Figure 5.17. Using the logic equations for MOSFETs gives the output as

$$g = \bar{A} \cdot \bar{B} \cdot 1 + (A + B) \cdot 0 \quad (5.39)$$

As in the case of the NAND gate, the nFET terms logically evaluate to 0 which leaves

$$\begin{aligned} g &= \bar{A} \cdot \bar{B} \\ &= \overline{(A + B)} \end{aligned} \quad (5.40)$$

using the DeMorgan theorem. This verifies our previous statement that the logic function is determined entirely by the topology of the circuit.

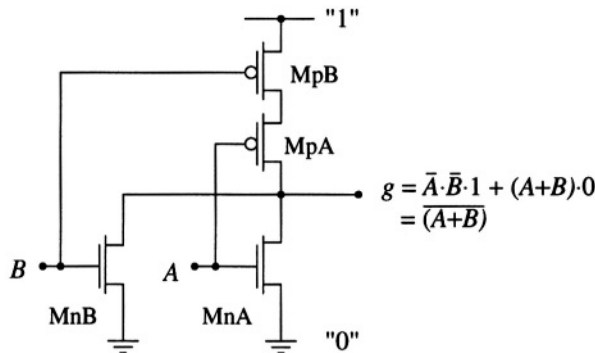


Figure 5.17 Logic operation of the CMOS NOR2 gate

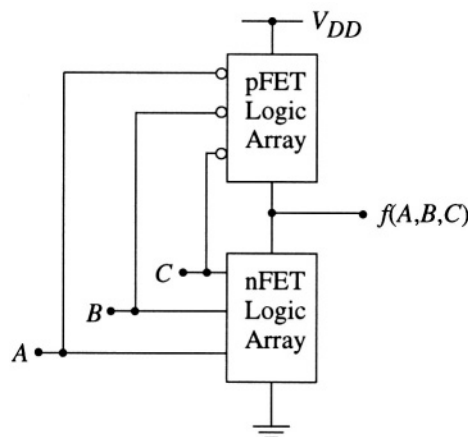


Figure 5.26 General structure of a CMOS logic gate

The NAND and NOR gates use series-parallel logic with opposite wiring for nFETs and pFETs.

5.4 Complex Logic Gates

Consider the general complex logic gate circuit reproduced for reference in Figure 5.26. As discussed in Section 5.1, the use of complementary pairs requires that every input drives both an nFET and a pFET. Complex logic functions can be implemented by designing the nFET and pFET switching arrays such that only one composite switch is closed for a given set of inputs. The switch equivalents are illustrated in Figure 5.27 to aid in visualizing the operation. If the nFET switch is closed while the pFET switch is open, then the output is a logic 0. Conversely, a closed pFET switch and an open nFET switch results in a logic 1 output. The arrays must be designed to avoid two situations: (1) both switches are open at the same time, since this gives an undetermined value;

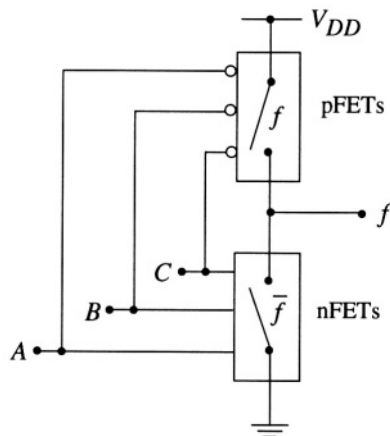


Figure 5.27 Switch operation of a general CMOS logic gate

and, (2) both are closed at the same time, since the voltage will not be a well-defined logic voltage. Note that we have labelled the nFET switching block by $\bar{f}$, the pFET block by f , and the output is also given as f . This notation is defined to mean that if the output f is a logical 1, the nFET block is OPEN and the pFET block is CLOSED. Conversely, if the nFET block is CLOSED while the pFET block is OPEN, the output is at a value of $f=0$ (i.e., $\bar{f} = 1$). With regards to the individual blocks themselves, the outcome ($\bar{f}$ for the nFETs and f for the pFETs) is TRUE (a logic 1) if the switch is closed.

A useful set of general logic formation rules can be obtained by extending the lessons learned from the study of the CMOS NAND2 and NOR2 circuits. Figure 5.28 summarizes how nFETs behave in the static gate circuit. When interpreting these sub-circuits, it is important to remember that (1) the source of the “lowest” nFET is connected to ground, and (2) the drain of the “highest” nFET is the output. Now then, recall that connecting nFETs in series gives the AND-NOT function, while wiring nFETs in parallel results in the OR-NOT operation; these are summarized in Figure 5.28(a) and (b). A set of general rules may be obtained by extending these results to groups of nFETs. In other words, the logic formation rules also apply if we replace the individual series- or parallel-connected transistors by blocks of transistors such as the groups illustrated in Figure 5.28(c) and (d) which respectively form the AND-OR-Invert (AOI) function

$$\overline{A \cdot B + C \cdot D} \quad (5.65)$$

and the OR-AND-Invert (OAI) expression

$$\overline{(A + B) \cdot (C + D)} \quad (5.66)$$

using the rules. These may be generalized to arbitrary nFET logic blocks.

Now let us examine the pFET rules. Figure 5.29(a) shows that two series-connected pFETs gives the OR-NOT operation, while the two parallel-connected pFETs in (b) results in the AND-

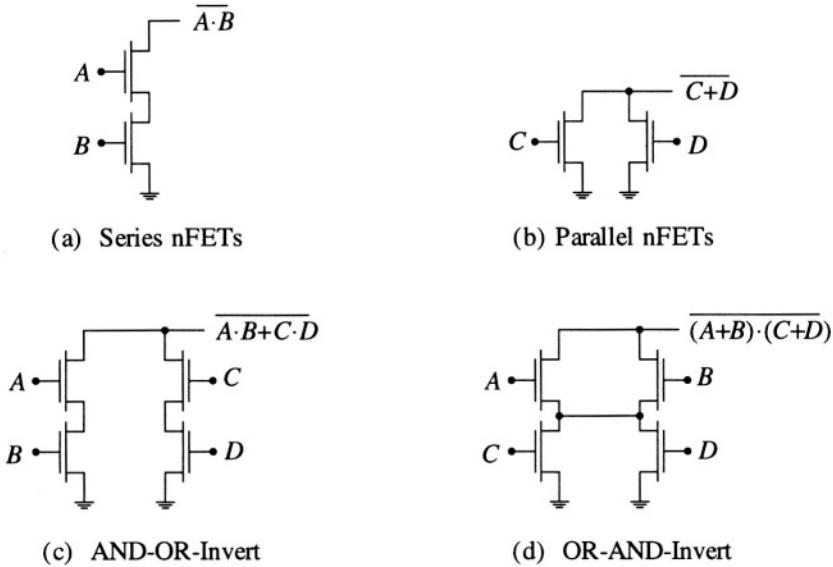


Figure 5.28 Logic formation using nFETs in the CMOS logic gate

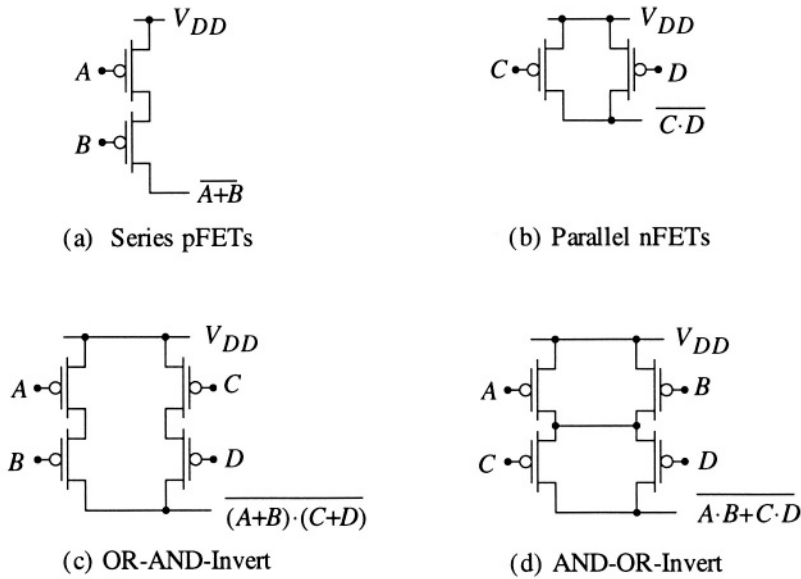


Figure 5.29 Logic formation using pFETs in the CMOS logic gate

NOT operation. This is exactly opposite to the manner in which nFETs behave. When the individual transistors are replaced by arrays of pFETs, we can create more complex functions. For example, in Figure 5.29(c), the series-connected pFETs implement the OR-AND-Invert operation

$$\overline{(A + B) \cdot (C + D)} \quad (5.67)$$

which is the dual of the equivalent nFET arrangement. Similarly, the sub-network in Figure 5.29(d) gives the AND-OR-Invert term

$$\overline{A \cdot B + C \cdot D} \quad (5.68)$$

A CMOS gate that is created with nFET/pFET groups using this type of structuring implements what is called **series-parallel logic**. Complex logic gates can be created by applying these rules to the complementary pairs; the resulting functions can be verified using switch logic formalism. When the nFET and pFET arrays are created in this manner, the resulting logic gates have a unique function assignment with well-defined outputs. This means that the DC output voltage is either V_{DD} or 0v; it is never left floating or simultaneously connected to both the power supply and ground.

5.4.1 Examples of Complex Logic Gates

Consider the circuit shown in Figure 5.30. This logic gate implements the function

$$f = \overline{A \cdot B + C} \quad (5.69)$$

since either $A \cdot B = 1$ OR $C = 1$ will connect the output to ground by turning on an nFET conduction path. This is verified by the function table shown with the gate. If $C = 0$ AND $(A = 0$ OR $B = 0)$, then the pFETs provide a conduction path between the power supply and the output, giving a logic 1 output voltage of V_{DD} . Note that the function has the AOI structuring discussed above. In particu-

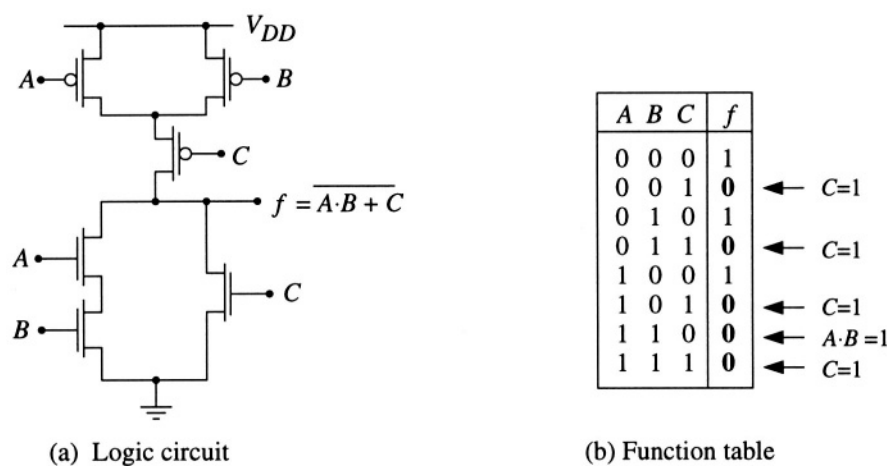


Figure 5.30 Example of a CMOS complex logic gate

lar, the nFETs with input variables A and B are in **series**, so the that pFETs with inputs A and B must be in **parallel**. Working outward in the nFET array we see that the nFET with input C is **parallel** with “ $A \cdot B$ ” group of nFETs, so that the C -pFET must be in **series** with the “ $A \cdot B$ ” group of pFETs. It is seen that this of structuring is very useful for design purposes. A logic gate can be designed by building the nFET logic array to meet the functional specifications. Once the nFET logic wiring is completed, the pFET array can be constructed by applying series-parallel arguments using the nFET connections. In general, the series-parallel design process must be initiated at the simplest sub-block level, and expanded outward until the entire array is in this form. For the above example, this meant that we first looked at the two series “ $A \cdot B$ ” nFETs, and then moved to the next level where the C -input nFET was in parallel with the “ $A \cdot B$ ” group.

The circuit shown in Figure 5.31 provides another example of a complex logic gate. This implements the function

$$g = \overline{A \cdot (B + C)} \quad (5.70)$$

as can be verified by the straightforward reasoning. Consider the viewpoint where we trace the output rail down to the ground connection. An input $A=1$ is needed to get through the “first level” of nFETs. To complete the path to ground, then either $B=1$ OR $C=1$ must be true. This says that the output node will be connected to ground if $A \cdot (B+C)=1$; however, this results in a logic 0 output (since the output node will be connected to ground), so that we must take the complement to arrive at the function g .

Another example of how a complex logic gate can be built is shown in Figure 5.32. Consider the nFET logic array. This group of transistors connects the output node to ground if $A \cdot (B+C)=1$, OR, if $D \cdot E=1$. Since either case gives an output of $f=0$, we may write by inspection that

$$f = \overline{A \cdot (B + C) + D \cdot E} \quad (5.71)$$

is the Boolean logic expression for the circuit. The series-parallel relationship between the nFET and the pFET arrays can be verified by starting with the smallest nFET subcircuit consisting of the “ $B+C$ ” parallel transistors, and then working outward with larger and larger groupings. Note that the function may be expanded to read as

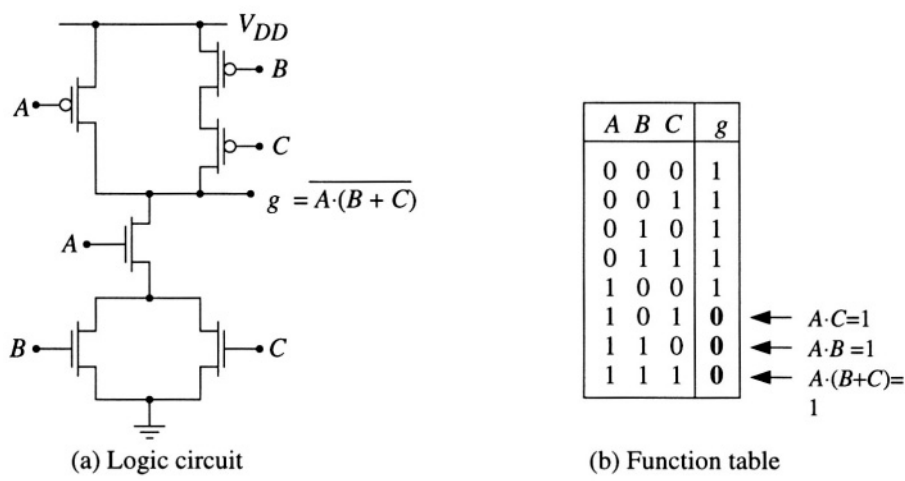


Figure 5.31 A complex logic CMOS gate circuit

$$f = \overline{A \cdot B + A \cdot C + D \cdot E} \tag{5.72}$$

which is true AOI form.

These examples should be studied in detail until the nature of the series-parallel nFET-pFET relationship becomes clear. Just remember to start with the largest FET groups, and then work into smaller and smaller groups until the gate is complete.

5.4.2 Logic Design Techniques

The examples above illustrate the technique for designing complex logic gates using function tables and/or occurrences of 0's at the output. Other approaches are easy to formulate. As an example, let us look at the problem of creating a complex logic gate from a logic diagram. Consider the

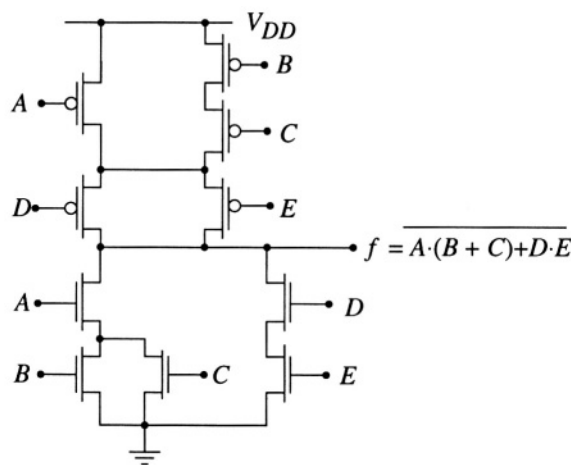


Figure 5.32 A 5-input complex logic gate

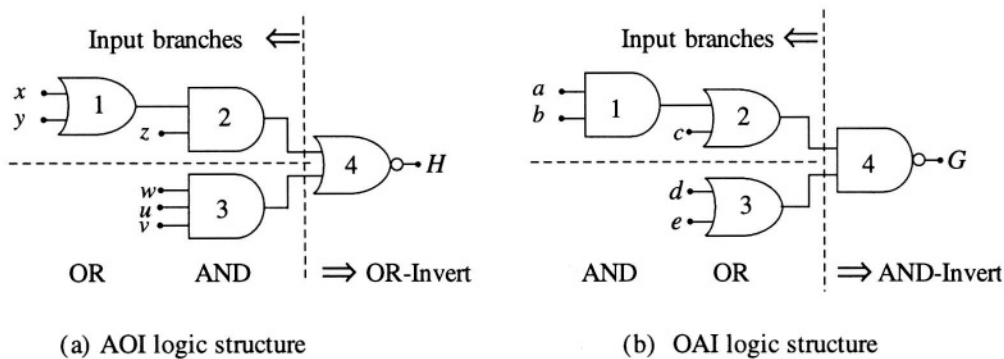


Figure 5.33 Basic logic cascades

logic diagrams shown in Figure 5.33. The AOI network (a) has been structured to show the logic patterning OR-AND-OR-Invert, while the OAI network (b) implements a flow of AND-OR-AND-Invert. The *Input Branches* in both drawings consist of the gates that have logic variables applied as inputs. These provide the basis for creating the CMOS logic circuit.

Consider first the logic characteristics of nFETs as summarized in Figure 5.34. In general, series-connected nFETs yield the AND operation among the inputs, while parallel-connected nFETs are used for ORing the input variables. This provides us with a one-to-one correspondence between the logic gate diagram and the structure of nFET logic array.

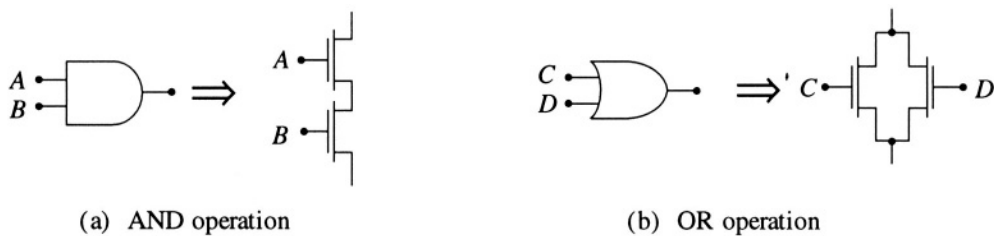


Figure 5.34 nFET-to-logic gate equivalence

The rules for pFETs are summarized in Figure 5.35, and requires a more detailed explanation. We will adopt the viewpoint that series-connected pFETs still yield the AND operation, but with **assert-low** (bubbled) inputs. For the pFETs shown in Figure 5.35(a), the series grouping corresponds to the AND operation

$$\overline{A} \cdot \overline{B} = \overline{A + B} \tag{5.73}$$

where the DeMorgan rule has been used to obtain the NOR equivalent. Applying the same reasoning to parallel-connected pFETs as shown in Figure 5.35(b) gives the OR operation with assert-low inputs. This is the same as the NAND operation since

$$\overline{A} + \overline{B} = \overline{A \cdot B} \tag{5.74}$$

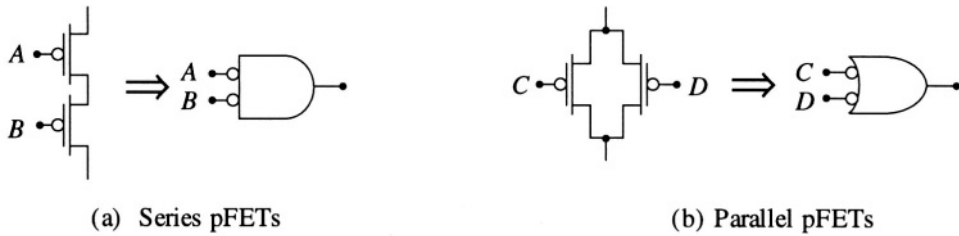


Figure 5.35 pFET-logic gate equivalence

by the basic DeMorgan identity. When the output of a gate is used as an input to another logic gate, it acts in the same manner. The AND operation implies that series groups of FETs are to be used, while OR is accomplished by means of parallel groups of FETs.

To understand how these structuring rules allow us to construct the required nFET and pFET logic arrays, let us examine the logic diagrams that were introduced in Figure 5.33. Since these have assert-high (normal or unbubbled inputs), we may use them directly to create the nFET logic arrays. The pFET circuit is based on assert-low inputs, so that it is helpful to modify the logic diagrams by “pushing the bubble from the output towards the input,” i.e., applying the DeMorgan theorems. This results in the logic diagrams shown in Figure 5.36. Since these have assert-low inputs, the pFET rules may be directly applied to arrive at the pFET logic array. This procedure results in the complex logic gates shown in Figure 5.37. It is a straightforward matter to verify the correspondence between the primitive logic operations and the resulting FET arrangements.

5.13 Problems

[5-3] Design the circuitry for a CMOS logic gate that gives the function

$$f = \overline{A \cdot (B + C) + B \cdot D}. \quad (5.173)$$

[5-4] Design a CMOS logic gate for the following functions

(a) $g = \overline{(A + B) \cdot (C + D) \cdot X}$

(b) $h = \overline{[x + (y \cdot z)] \cdot w}$

(c) $u = \overline{x + y + z \cdot w}$

[5-5] Use the function table in Figure P5.1 to find f . Then design a CMOS gate for this function.

A	B	C	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Figure P5.1

x	y	z	g
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Figure P5.2

u	v	w	T
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Figure P5.3

[5-6] Use the function table in Figure P5.2 to find g . Then design a CMOS gate for this function.

[5-7] Use the function table in Figure P5.3 to find T . Then design a CMOS gate for this function.